

Frontend Performance Checklist

Comprehensive checklist covering page weight, load time, JavaScript and CSS optimization, image optimization, Core Web Vitals, CDN and network strategies, service workers, rendering performance, font loading, resource hints, and performance tooling. Prioritized from a full-stack / team lead perspective.

Sources: Items labeled "Screenshot" are derived from the roadmap.sh Frontend Performance checklist. Items labeled "Best practices" are additional recommendations that complement those topics. This document is self-contained and may overlap with other checklists in this series.

HIGH PRIORITY

Address first. These have the biggest direct impact on load time, user experience, and Core Web Vitals: page weight, compression, JavaScript/CSS blocking, image optimization, and key performance metrics.

☐	Checklist Item	Source
Page Weight & Load Time		
☐	Keep your page weight under 1500 KB (ideally under 500 KB)	Screenshot
☐	Keep your page load time under 3 seconds	Screenshot
☐	Keep the Time To First Byte (TTFB) under 1.3 seconds	Screenshot
☐	Minimize HTTP requests by combining files, using sprites, or eliminating unnecessary resources	Screenshot
☐	Avoid requesting unreachable files (404s) — audit and fix broken resource references	Screenshot
☐	Serve files from the same protocol (HTTPS everywhere) to avoid mixed content issues	Screenshot
Compression & Transfer		
☐	Enable GZIP or Brotli compression for all text-based resources (HTML, CSS, JS, SVG, JSON)	Screenshot
☐	Use HTTPS on your entire website	Screenshot
☐	Set HTTP cache headers properly: immutable hashed assets with long max-age; short cache for HTML	Screenshot
JavaScript Optimization		
☐	Use non-blocking JavaScript: add async or defer attributes to script tags	Screenshot

☐	Checklist Item	Source
☐	Minify your JavaScript — remove whitespace, comments, and shorten variable names in production builds	Screenshot
☐	Avoid multiple inline JavaScript snippets (<script> blocks in the HTML body)	Screenshot
☐	Check for performance problems in your JavaScript files (long tasks, memory leaks, excessive DOM manipulation)	Screenshot
☐	Keep your dependencies up to date to benefit from performance fixes and smaller bundle sizes	Screenshot
☐	Code-split your JavaScript bundles so users only download code for the current route/feature	Best practices
☐	Tree-shake unused exports from your bundles using your bundler's dead-code elimination	Best practices
☐	Avoid long-running main-thread tasks (>50ms); break them up with requestIdleCallback, setTimeout, or web workers	Best practices
CSS Optimization		
☐	Minify your CSS — remove comments, whitespace, and unnecessary characters	Screenshot
☐	Inline the Critical CSS (above-the-fold CSS) directly in the <head> to eliminate render-blocking	Screenshot
☐	Ensure CSS files are non-blocking: use media attributes or load non-critical CSS asynchronously	Screenshot
☐	Avoid excessive embedded / inline CSS scattered throughout the HTML body	Screenshot
☐	Analyze stylesheet complexity — reduce deeply nested selectors, unused rules, and overly specific chains	Screenshot
Image Optimization		
☐	Compress your images and keep the image count low; remove unnecessary decorative images	Screenshot
☐	Choose your image format appropriately: WebP/AVIF for photos, SVG for icons/illustrations, PNG for transparency when needed	Screenshot
☐	Set width and height attributes on images (aspect ratio) to prevent layout shift during loading	Screenshot
☐	Ensure offscreen images are loaded lazily (loading="lazy" attribute or Intersection Observer)	Screenshot
☐	Serve images that are close to your display size — don't serve 4000px images for 400px containers	Screenshot

☐	Checklist Item	Source
☐	Prefer using vector images (SVG) rather than bitmap images where appropriate	Screenshot
☐	Avoid using Base64 images (they increase page weight by ~33% and can't be cached independently)	Screenshot
☐	Use responsive images with srcset and sizes attributes to serve appropriate resolutions per viewport	Best practices
☐	Minimize number of iframes — each iframe is an additional document load with its own resources	Screenshot
Core Web Vitals		
☐	Measure and optimize Largest Contentful Paint (LCP) — target under 2.5 seconds	Best practices
☐	Measure and optimize Interaction to Next Paint (INP) — target under 200ms	Best practices
☐	Measure and minimize Cumulative Layout Shift (CLS) — target under 0.1	Best practices
☐	Monitor Core Web Vitals with Real User Monitoring (RUM) in production, not just lab tools	Best practices



MEDIUM PRIORITY

Tackle once high-priority items are solid. Covers HTML optimization, CDN and network, service workers, cookies, third-party script management, rendering/paint performance, and resource hints.

☐	Checklist Item	Source
HTML Optimization		
☐	Minify HTML — remove comments and unnecessary whitespace from production HTML	Screenshot
☐	Use semantic HTML elements to reduce unnecessary wrapper divs and keep the DOM tree lean	Best practices
☐	Keep the DOM node count reasonable (under ~1500 nodes); deep/wide DOM trees slow rendering and JS performance	Best practices
CDN & Network		
☐	Use a Content Delivery Network (CDN) to serve static assets from edge locations near your users	Screenshot
☐	Use preconnect for critical third-party origins to establish early connections (<link rel="preconnect">)	Best practices
☐	Use DNS prefetch for non-critical third-party domains (<link rel="dns-prefetch">)	Best practices
☐	Enable HTTP/2 or HTTP/3 on your server/CDN for multiplexed connections and header compression	Best practices
Service Workers & Caching		
☐	Use Service Workers for caching strategies (cache-first, stale-while-revalidate) and performing heavy tasks offline	Screenshot
☐	Implement a service worker precaching strategy for critical app shell resources	Best practices
☐	Handle service worker updates gracefully — notify users of new versions and prompt reload	Best practices
Cookies & Storage		
☐	Keep cookie size under 4096 bytes per cookie	Screenshot
☐	Keep the total cookie count under 20 per domain	Screenshot
☐	Audit cookies regularly: remove expired or unnecessary cookies; set appropriate SameSite, Secure, and HttpOnly flags	Best practices
Third-Party Scripts & Dependencies		
☐	Audit and minimize third-party JavaScript: analytics, ads, widgets, chat, and tag managers all add weight	Best practices

☐	Checklist Item	Source
☐	Load third-party scripts asynchronously or defer them to prevent blocking the main thread	Best practices
☐	Use Subresource Integrity (SRI) for all third-party scripts and stylesheets loaded from CDNs	Best practices
☐	Keep an eye on the size of your dependencies using tools like Bundlephobia before adding new packages	Screenshot
☐	Regularly audit bundle composition with webpack-bundle-analyzer or similar to find bloat	Best practices
Rendering & Paint Performance		
☐	Avoid layout thrashing: batch DOM reads and writes rather than interleaving them	Best practices
☐	Use CSS containment (contain: layout, paint, size) on isolated components to limit browser recalculation scope	Best practices
☐	Use will-change or transform: translateZ(0) sparingly on elements that will animate to promote them to GPU layers	Best practices
☐	Prefer CSS animations and transitions over JavaScript-driven animation for smoother, compositor-thread performance	Best practices
☐	Use content-visibility: auto on long pages to skip rendering of off-screen content entirely	Best practices
☐	Avoid forced synchronous layouts by not reading layout properties (offsetHeight, getBoundingClientRect) after DOM writes	Best practices
Resource Hints & Preloading		
☐	Preload critical resources (hero image, above-the-fold fonts, key LCP assets) with <link rel="preload">	Best practices
☐	Use fetchpriority="high" on the LCP image or critical above-the-fold resources	Best practices
☐	Use fetchpriority="low" on below-the-fold images and non-critical resources to deprioritize them	Best practices



LOW PRIORITY

Valuable for squeezing out maximum performance. Covers CSS refinement, font optimization, dependency/bundle management, advanced loading strategies, and performance tools and monitoring.

☐	Checklist Item	Source
CSS Refinement		
☐	Pre-load URLs where possible using <code><link rel="prefetch"></code> for anticipated next-page navigations	Screenshot
☐	Concatenate CSS into a single file (or minimal files) to reduce HTTP requests in HTTP/1.1 environments	Screenshot
☐	Remove unused CSS using tools like PurgeCSS or the Coverage tab in Chrome DevTools	Screenshot
☐	Analyze and reduce CSS specificity wars — use methodologies like BEM or utility-first CSS for maintainability	Best practices
Font Optimization		
☐	Use WOFF2 font format (smallest file size with broad browser support)	Screenshot
☐	Use <code><link rel="preconnect"></code> to load your fonts faster from external font services	Screenshot
☐	Keep the total web font size under 300 KB across all weights and styles	Screenshot
☐	Prevent Flash of Invisible Text (FOIT) or Flash of Unstyled Text (FOUT) using font-display: swap or optional	Screenshot
☐	Subset fonts to include only the character ranges you actually use (e.g., Latin only)	Best practices
☐	Self-host fonts when possible to eliminate third-party DNS lookups and reduce latency	Best practices
Dependency & Bundle Management		
☐	Keep an eye on the size of dependencies; audit regularly with Bundlephobia before adding new packages	Screenshot
☐	Replace heavy libraries with lighter alternatives where possible (e.g., date-fns instead of moment.js)	Best practices
☐	Use dynamic imports (import()) for route-level and feature-level code splitting	Best practices
☐	Set up bundle size budgets in your build tool and fail CI if thresholds are exceeded	Best practices
Advanced Loading Strategies		

☐	Checklist Item	Source
☐	Implement predictive prefetching based on user navigation patterns (e.g., guess.js or quicklink)	Best practices
☐	Use the Speculation Rules API for instant page transitions on anticipated navigations	Best practices
☐	Implement progressive hydration or partial hydration for SSR frameworks to reduce time-to-interactive	Best practices
☐	Consider using Islands Architecture (Astro) for content-heavy sites that need minimal interactivity	Best practices
Performance Tools & Monitoring		
☐	Use PageSpeed Insights for field data (CrUX) and lab data combined assessment	Screenshot
☐	Use Lighthouse (via DevTools or CI) for detailed performance, accessibility, SEO, and best practices audits	Screenshot
☐	Use WebPageTest for waterfall analysis, filmstrip views, and multi-location performance testing	Screenshot
☐	Use Chrome DevTools Performance panel for flame charts, main-thread analysis, and layout shift debugging	Screenshot
☐	Use Bundlephobia to check package sizes before adding new npm dependencies	Screenshot
☐	Use Squoosh.app or similar tools for manual image compression and format conversion	Screenshot
☐	Set up continuous performance monitoring with SpeedCurve, Calibre, or web-vitals library reporting to your analytics	Best practices
☐	Run performance audits on slow network conditions (3G throttling) to catch issues that only appear on constrained connections	Best practices

Generated March 2026. Based on roadmap.sh Frontend Performance checklist and industry best practices.